SENATI

Escuela de Tecnologías de la Información

SOLUCIONARIO

Solucionario Guía 16

Entropía y el Algoritmo ID3

Solución paso a paso de los ejercicios propuestos y de las preguntas de reflexión

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Artificial Intelligence with Machine Learning in Java (Oracle Academy)
Semestre	IV
Semana	16
Documento	Solucionario de los ejercicios propuestos de la Guía 16
Herramientas	Java 21 (JDK) · Visual Studio Code · Extension Pack for Java
Instructor	Rembrandt Ubalde Enríquez
Uso	Material del instructor

Contenido

Indicaciones generales 3
Ejercicio 1. Entropía calculada a mano 4
Paso 1. Cálculos a mano 4
Paso 2. Verificación en Java 4
Ejercicio 2. Ganancia de información de los créditos 6
Paso 1. Qué hay que cambiar 6
Paso 2. Programa 6
Paso 3. Respuesta explicada 7
Ejercicio 3. Agregar un día 15 al dataset del tenis 8
Paso 1. Programa 8
Paso 2. Comparación de los dos árboles 10
Paso 3. Explicación 10
Ejercicio 4. Árbol ID3 con un dataset propio 11
Paso 1. Crear el dataset 11
Paso 2. Programa 11
Paso 3. Interpretación 12
Ejercicio 5. Matriz de confusión del evaluador de créditos 14
Paso 1. Conceptos 14
Paso 2. Programa 14
Paso 3. Respuesta explicada 15
Respuestas a las preguntas de reflexión 17

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Indicaciones generales

Cada ejercicio se resuelve en un archivo nuevo dentro de la carpeta `src` del proyecto **Semana16_ID3_Java**. Los nombres de los archivos empiezan con `Ej` seguido del número del ejercicio, para no confundirlos con las prácticas.

Ejercicio	Archivo(s)	Necesita
1	`Ej01_EntropiaManual.java`	—
2	`Ej02_GananciaCreditos.java`	`Datos`, `ID3`, `NodoID3` y `datos/creditos.csv`
3	`Ej03_TenisDia15.java`	`Datos`, `ID3`, `NodoID3` y `datos/jugar_tenis.csv`
4	`Ej04_Aprobacion.java` y `datos/aprobacion.csv`	`Datos`, `ID3`, `NodoID3`
5	`Ej05_MatrizConfusion.java`	`Datos`, `ID3`, `NodoID3` y `datos/creditos.csv`

Todos los resultados de este solucionario se obtuvieron **ejecutando el código con Java 21**; no son aproximados. Para usar `ID3.entropia(...)` desde otra clase, el método es `public static` en `ID3.java`, tal como está en el proyecto de la guía.

Ejercicio 1. Entropía calculada a mano

Enunciado: Calcula a mano la entropía de los conjuntos [6 Sí, 2 No], [4 Sí, 4 No] y [3, 3, 2] (tres clases). Verifica tus resultados en Java.

Paso 1. Cálculos a mano

Conjunto [6, 2] (8 elementos)

```
p1 = 6/8 = 0.75     -p1·log2(p1) = -0.75 · (-0.415) = 0.311
p2 = 2/8 = 0.25     -p2·log2(p2) = -0.25 · (-2.000) = 0.500
H = 0.311 + 0.500 = 0.811
```

Conjunto [4, 4] (8 elementos)

```
p1 = p2 = 0.5       -0.5 · log2(0.5) = -0.5 · (-1) = 0.5
H = 0.5 + 0.5 = 1.000     (máximo desorden con 2 clases)
```

Conjunto [3, 3, 2] (8 elementos)

```
p1 = p2 = 3/8 = 0.375   -0.375 · log2(0.375) = -0.375 · (-1.415) = 0.531  (dos veces)
p3 = 2/8 = 0.25         -0.25 · log2(0.25)  = 0.500
H = 0.531 + 0.531 + 0.500 = 1.561
```

> 💡 **Consejo:** Para calcular $\log_2$ con una calculadora que solo tiene **ln** o **log**: $\log_2(x) = \ln(x) / \ln(2) = \log(x) / \log(2)$.

Paso 2. Verificación en Java

Archivo: src/Ej01_EntropiaManual.java

```
// Ej01_EntropiaManual.java - Verificación de los cálculos hechos a mano
import java.util.Arrays;
import java.util.Locale;

public class Ej01_EntropiaManual {

  static double entropia(int... conteos) {
    int total = Arrays.stream(conteos).sum();
    double h = 0;
    for (int c : conteos) {
      if (c == 0) continue;
      double p = (double) c / total;
      h -= p * Math.log(p) / Math.log(2);
    }
    return h;
  }

  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    int[][] casos = {{6, 2}, {4, 4}, {3, 3, 2}};
    for (int[] c : casos) {
      System.out.printf("%-10s -> H = %.3f bits%n", Arrays.toString(c), entropia(c));
    }
  }
}
```

▶ **Resultado esperado en la terminal**

```
[6, 2]     -> H = 0.811 bits
[4, 4]     -> H = 1.000 bits
[3, 3, 2]  -> H = 1.561 bits
```

Análisis: con tres clases la entropía puede **superar 1** (el máximo es $\log_2(3) = 1.585$). El conjunto [3, 3, 2] está casi en ese máximo porque sus tres clases están casi igual de repartidas.

Ejercicio 2. Ganancia de información de los créditos

Enunciado: Adapta la Práctica 02 para calcular la ganancia de los 4 atributos de `creditos.csv` con las 24 filas. ¿Qué atributo sería la raíz? ¿Coincide con el árbol de la Práctica 04?

Paso 1. Qué hay que cambiar

- El nombre del archivo: `datos/creditos.csv`.
- La Práctica 02 contaba «Sí» y «No» y escribía `/14`. Para que sirva con **cualquier** dataset, ahora se cuenta cada clase con un `Map` y se usa `ID3.entropia(...)`, que ya calcula la entropía de cualquier grupo de filas.

Paso 2. Programa

Archivo: src/Ej02_GananciaCreditos.java

```
// Ej02_GananciaCreditos.java - Ganancia de información de creditos.csv (24 filas)
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;

public class Ej02_GananciaCreditos {
  public static void main(String[] args) throws IOException {
    Locale.setDefault(Locale.US);
    Datos d = Datos.cargarCSV("datos/creditos.csv");
    double hTotal = ID3.entropia(d.filas);
    System.out.printf("Filas: %d | H(S) = %.3f%n%n", d.filas.size(), hTotal);

    String mejor = null;
    double mejorG = -1;
    for (String atributo : d.atributos) {
      int col = d.columna(atributo);
      Map<String, List<String[]>> grupos = new TreeMap<>();
      for (String[] f : d.filas) {
        grupos.computeIfAbsent(f[col], k -> new ArrayList<>()).add(f);
      }
      double hPond = 0;
      StringBuilder detalle = new StringBuilder();
      for (var g : grupos.entrySet()) {
        // Conteo de cada clase dentro del grupo (sirve para cualquier dataset)
        Map<String, Integer> conteo = new TreeMap<>();
        for (String[] f : g.getValue()) conteo.merge(Datos.clase(f), 1, Integer::sum);
        double h = ID3.entropia(g.getValue());
        hPond += (double) g.getValue().size() / d.filas.size() * h;
        detalle.append(String.format("   %-9s %2d filas %s  H = %.3f%n",
            g.getKey(), g.getValue().size(), conteo, h));
      }
      double ganancia = hTotal - hPond;
      System.out.printf("%s%n%s   Ganancia = %.3f%n%n", atributo, detalle, ganancia);
      if (ganancia > mejorG) { mejorG = ganancia; mejor = atributo; }
    }
    System.out.printf("RAÍZ con las 24 filas: %s (ganancia = %.3f)%n", mejor, mejorG);
  }
}
```

▶ **Resultado esperado en la terminal**

```
Filas: 24 | H(S) = 1.000

Ingreso
   Alto       8 filas {No=2, Sí=6}  H = 0.811
   Bajo       8 filas {No=6, Sí=2}  H = 0.811
   Medio      8 filas {No=4, Sí=4}  H = 1.000
   Ganancia = 0.126

Deudas
   No        12 filas {No=3, Sí=9}  H = 0.811
   Sí        12 filas {No=9, Sí=3}  H = 0.811
   Ganancia = 0.189

Empleo
   Estable   12 filas {No=5, Sí=7}  H = 0.980
   Temporal  12 filas {No=7, Sí=5}  H = 0.980
   Ganancia = 0.020

Historial
   Bueno     12 filas {No=3, Sí=9}  H = 0.811
   Malo      12 filas {No=9, Sí=3}  H = 0.811
   Ganancia = 0.189

RAÍZ con las 24 filas: Deudas (ganancia = 0.189)
```

Paso 3. Respuesta explicada

- El conjunto completo tiene 12 Sí y 12 No, por eso H(S) = **1.000** (máximo desorden).
- **Deudas** e **Historial** empatan con la mayor ganancia (**0.189**). El programa elige **Deudas** porque aparece primero en el CSV y la comparación es `ganancia > mejor` (estrictamente mayor). Con `>=` se elegiría Historial; ambas raíces son igual de válidas.
- **Empleo** casi no aporta (0.020): los aprobados y denegados se reparten casi igual entre empleo estable y temporal.
- **¿Coincide con la Práctica 04?** Sí, la raíz también fue Deudas. Pero **podría ser diferente**, porque la Práctica 04 entrena solo con 18 de las 24 filas. Con otra división (otra semilla) Historial podría quedar por encima de Deudas y la raíz cambiaría.

Ejercicio 3. Agregar un día 15 al dataset del tenis

Enunciado: Agrega al archivo `jugar_tenis.csv` el día 15: `Nublado, Templado, Alta, Débil, No`. Vuelve a ejecutar la Práctica 03. ¿Cómo cambia el árbol?

Paso 1. Programa

Para no modificar el archivo original, el día 15 se agrega en memoria con `datos.filas.add(...)`. El resultado es el mismo que si se agregara la línea al CSV.

Archivo: src/Ej03_TenisDia15.java

```
// Ej03_TenisDia15.java - ¿Qué pasa con el árbol si agregamos un día nuevo?
import java.io.IOException;

public class Ej03_TenisDia15 {
  public static void main(String[] args) throws IOException {
    Datos datos = Datos.cargarCSV("datos/jugar_tenis.csv");
    // Día 15: Nublado, Templado, Alta, Débil -> No
    datos.filas.add(new String[]{"Nublado", "Templado", "Alta", "Débil", "No"});
    System.out.println("Días: " + datos.filas.size());

    ID3 id3 = new ID3(datos);
    id3.mostrarPasos = true;
    id3.entrenar();
    System.out.println("\nÁrbol con 15 días:");
    id3.imprimir();
    System.out.println("\nReglas:");
    id3.imprimirReglas();
  }
}
```

▶ Resultado esperado en la terminal

```
Días: 15
Ganancia(Cielo) = 0.083
Ganancia(Temperatura) = 0.028
Ganancia(Humedad) = 0.186
Ganancia(Viento) = 0.020
=> se elige: Humedad
Rama Humedad = Alta (8 filas)
   Ganancia(Cielo) = 0.360
   Ganancia(Temperatura) = 0.003
   Ganancia(Viento) = 0.003
   => se elige: Cielo
   Rama Cielo = Lluvioso (2 filas)
      Ganancia(Temperatura) = 0.000
      Ganancia(Viento) = 1.000
      => se elige: Viento
      Rama Viento = Débil (1 filas)
      Rama Viento = Fuerte (1 filas)
   Rama Cielo = Nublado (3 filas)
      Ganancia(Temperatura) = 0.252
      Ganancia(Viento) = 0.252
      => se elige: Temperatura
      Rama Temperatura = Caluroso (1 filas)
      Rama Temperatura = Templado (2 filas)
         Ganancia(Viento) = 1.000
         => se elige: Viento
         Rama Viento = Débil (1 filas)
         Rama Viento = Fuerte (1 filas)
   Rama Cielo = Soleado (3 filas)
Rama Humedad = Normal (7 filas)
   Ganancia(Cielo) = 0.198
   Ganancia(Temperatura) = 0.128
   Ganancia(Viento) = 0.198
   => se elige: Cielo
   Rama Cielo = Lluvioso (3 filas)
      Ganancia(Temperatura) = 0.252
      Ganancia(Viento) = 0.918
      => se elige: Viento
      Rama Viento = Débil (2 filas)
      Rama Viento = Fuerte (1 filas)
   Rama Cielo = Nublado (2 filas)
   Rama Cielo = Soleado (2 filas)

Árbol con 15 días:
[Humedad]
├── Alta [Cielo]
│   ├── Lluvioso [Viento]
│   │   ├── Débil => Jugar: SÍ
│   │   └── Fuerte => Jugar: NO
│   ├── Nublado [Temperatura]
│   │   ├── Caluroso => Jugar: SÍ
│   │   └── Templado [Viento]
│   │       ├── Débil => Jugar: NO
│   │       └── Fuerte => Jugar: SÍ
│   └── Soleado => Jugar: NO
└── Normal [Cielo]
    ├── Lluvioso [Viento]
    │   ├── Débil => Jugar: SÍ
    │   └── Fuerte => Jugar: NO
    ├── Nublado => Jugar: SÍ
    └── Soleado => Jugar: SÍ

Reglas:
SI Humedad = Alta Y Cielo = Lluvioso Y Viento = Débil ENTONCES Jugar = Sí
SI Humedad = Alta Y Cielo = Lluvioso Y Viento = Fuerte ENTONCES Jugar = No
SI Humedad = Alta Y Cielo = Nublado Y Temperatura = Caluroso ENTONCES Jugar = Sí
SI Humedad = Alta Y Cielo = Nublado Y Temperatura = Templado Y Viento = Débil ENTONCES Jugar = No
SI Humedad = Alta Y Cielo = Nublado Y Temperatura = Templado Y Viento = Fuerte ENTONCES Jugar = Sí
SI Humedad = Alta Y Cielo = Soleado ENTONCES Jugar = No
SI Humedad = Normal Y Cielo = Lluvioso Y Viento = Débil ENTONCES Jugar = Sí
SI Humedad = Normal Y Cielo = Lluvioso Y Viento = Fuerte ENTONCES Jugar = No
```

```
SI Humedad = Normal Y Cielo = Nublado ENTONCES Jugar = Sí
SI Humedad = Normal Y Cielo = Soleado ENTONCES Jugar = Sí
```

Paso 2. Comparación de los dos árboles

	14 días (Práctica 03)	15 días (este ejercicio)
Raíz	Cielo (ganancia 0.247)	**Humedad** (ganancia 0.186)
Ganancia de Cielo	0.247	0.083
Número de reglas	5	**10**
Temperatura	No aparece	Aparece en una rama

Paso 3. Explicación

- Antes, la rama **Nublado** era pura (4 Sí, 0 No) y eso le daba mucha ganancia a Cielo. El día 15 es Nublado con «No», así que esa rama deja de ser pura y la ganancia de Cielo cae de 0.247 a 0.083.
- Ahora **Humedad** separa mejor los datos y pasa a ser la raíz. El árbol completo se reorganiza y crece al doble de reglas.
- **Conclusión:** un solo ejemplo nuevo puede cambiar todo el árbol. Los árboles de decisión son **inestables** frente a pequeños cambios en los datos o frente a datos con ruido. Por eso en la práctica se usan técnicas como la **poda** o se combinan muchos árboles (**Random Forest**).

Ejercicio 4. Árbol ID3 con un dataset propio

Enunciado: Crea tu propio dataset `aprobacion.csv` con al menos 14 filas y los atributos Asistencia (Alta/Media/Baja), Tareas (Completas/Incompletas), Estudia (Sí/No) y la clase Aprueba (Sí/No). Construye el árbol con ID3 y muestra sus reglas.

Paso 1. Crear el dataset

Se registraron 16 estudiantes. Es una solución de ejemplo: cada estudiante puede crear sus propios datos, y su árbol será distinto.

Archivo: datos/aprobacion.csv

```
Asistencia,Tareas,Estudia,Aprueba
Alta,Completas,Sí,Sí
Alta,Completas,No,Sí
Alta,Incompletas,Sí,Sí
Alta,Incompletas,No,No
Media,Completas,Sí,Sí
Media,Completas,No,Sí
Media,Incompletas,Sí,Sí
Media,Incompletas,No,No
Baja,Completas,Sí,Sí
Baja,Completas,No,No
Baja,Incompletas,Sí,No
Baja,Incompletas,No,No
Alta,Completas,Sí,Sí
Media,Incompletas,No,No
Baja,Completas,Sí,Sí
Baja,Incompletas,No,No
```

Paso 2. Programa

La clase `ID3` no se modifica: solo cambia el archivo de datos.

Archivo: src/Ej04_Aprobacion.java

```
// Ej04_Aprobacion.java - Árbol ID3 con un dataset propio: ¿el estudiante aprueba?
import java.io.IOException;

public class Ej04_Aprobacion {
  public static void main(String[] args) throws IOException {
    Datos datos = Datos.cargarCSV("datos/aprobacion.csv");
    System.out.println("Estudiantes: " + datos.filas.size()
        + " | Atributos: " + datos.atributos);

    ID3 id3 = new ID3(datos);
    id3.mostrarPasos = true;
    id3.entrenar();
    System.out.println("\nÁrbol de decisión:");
    id3.imprimir();
    System.out.println("\nReglas:");
    id3.imprimirReglas();

    String[][] nuevos = {{"Media", "Completas", "No"}, {"Baja", "Incompletas", "Sí"}};
    System.out.println("\nPredicciones:");
    for (String[] e : nuevos) {
      System.out.println("  " + String.join(", ", e)
          + " -> ¿Aprueba? " + id3.clasificar(e));
    }
  }
}
```

▶ Resultado esperado en la terminal

```
Estudiantes: 16 | Atributos: [Asistencia, Tareas, Estudia]
Ganancia(Asistencia) = 0.115
Ganancia(Tareas) = 0.311
Ganancia(Estudia) = 0.311
=> se elige: Tareas
Rama Tareas = Completas (8 filas)
   Ganancia(Asistencia) = 0.199
   Ganancia(Estudia) = 0.199
   => se elige: Asistencia
   Rama Asistencia = Alta (3 filas)
   Rama Asistencia = Baja (3 filas)
      Ganancia(Estudia) = 0.918
      => se elige: Estudia
      Rama Estudia = No (1 filas)
      Rama Estudia = Sí (2 filas)
   Rama Asistencia = Media (2 filas)
Rama Tareas = Incompletas (8 filas)
   Ganancia(Asistencia) = 0.217
   Ganancia(Estudia) = 0.467
   => se elige: Estudia
   Rama Estudia = No (5 filas)
   Rama Estudia = Sí (3 filas)
      Ganancia(Asistencia) = 0.918
      => se elige: Asistencia
      Rama Asistencia = Alta (1 filas)
      Rama Asistencia = Baja (1 filas)
      Rama Asistencia = Media (1 filas)

Árbol de decisión:
[Tareas]
├── Completas [Asistencia]
│    ├── Alta => Aprueba: SÍ
│    ├── Baja [Estudia]
│    │    ├── No => Aprueba: NO
│    │    └── Sí => Aprueba: SÍ
│    └── Media => Aprueba: SÍ
└── Incompletas [Estudia]
     ├── No => Aprueba: NO
     └── Sí [Asistencia]
          ├── Alta => Aprueba: SÍ
          ├── Baja => Aprueba: NO
          └── Media => Aprueba: SÍ

Reglas:
SI Tareas = Completas Y Asistencia = Alta ENTONCES Aprueba = Sí
SI Tareas = Completas Y Asistencia = Baja Y Estudia = No ENTONCES Aprueba = No
SI Tareas = Completas Y Asistencia = Baja Y Estudia = Sí ENTONCES Aprueba = Sí
SI Tareas = Completas Y Asistencia = Media ENTONCES Aprueba = Sí
SI Tareas = Incompletas Y Estudia = No ENTONCES Aprueba = No
SI Tareas = Incompletas Y Estudia = Sí Y Asistencia = Alta ENTONCES Aprueba = Sí
SI Tareas = Incompletas Y Estudia = Sí Y Asistencia = Baja ENTONCES Aprueba = No
SI Tareas = Incompletas Y Estudia = Sí Y Asistencia = Media ENTONCES Aprueba = Sí

Predicciones:
  Media, Completas, No -> ¿Aprueba? Sí
  Baja, Incompletas, Sí -> ¿Aprueba? No
```

Paso 3. Interpretación

- **Tareas** y **Estudia** empatan en la raíz (0.311); se elige Tareas por aparecer primero en el CSV.
- **Asistencia** es el atributo menos útil en la raíz (0.115), pero dentro de algunas ramas sí ayuda a decidir.
- Lectura de una regla: «SI Tareas = Incompletas Y Estudia = No ENTONCES Aprueba = No». Los estudiantes que no entregan tareas y no estudian desaprueban, sin importar su asistencia.

> ⚠ **Atención:** Si el dataset tiene filas **contradictorias** (mismos atributos, distinta clase), ID3 se queda sin atributos y la hoja toma la clase **mayoritaria**. Es correcto, pero conviene revisar esos datos.

Ejercicio 5. Matriz de confusión del evaluador de créditos

Enunciado: Modifica la Práctica 04 para mostrar la matriz de confusión. ¿Qué error es más costoso para la cooperativa?

Paso 1. Conceptos

Sigla	Nombre	Significado en este problema
VP	Verdadero positivo	Debía aprobarse y el árbol **aprobó**.
VN	Verdadero negativo	Debía denegarse y el árbol **denegó**.
FP	Falso positivo	Debía denegarse, pero el árbol **aprobó** (riesgo de no pago).
FN	Falso negativo	Debía aprobarse, pero el árbol **denegó** (cliente perdido).

Paso 2. Programa

Se usa la misma división de la Práctica 04 (`new Random(7)`, 75 % / 25 %), así que se evalúan los mismos 6 clientes.

Archivo: src/Ej05_MatrizConfusion.java

```
// Ej05_MatrizConfusion.java - Matriz de confusión del evaluador de créditos
import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Random;

public class Ej05_MatrizConfusion {
  public static void main(String[] args) throws IOException {
    Locale.setDefault(Locale.US);
    Datos todos = Datos.cargarCSV("datos/creditos.csv");
    List<String[]> filas = new ArrayList<>(todos.filas);
    Collections.shuffle(filas, new Random(7));              // misma división de la P04
    int corte = (int) (filas.size() * 0.75);
    Datos train = new Datos(todos.atributos, todos.nombreClase,
        filas.subList(0, corte));
    List<String[]> test = filas.subList(corte, filas.size());

    ID3 id3 = new ID3(train);
    id3.entrenar();

    // Clase positiva = "Sí" (crédito aprobado)
    int vp = 0, vn = 0, fp = 0, fn = 0;
    for (String[] f : test) {
      boolean real = Datos.clase(f).equals("Sí");
      boolean pred = id3.clasificar(f).equals("Sí");
      if (real && pred) vp++;
      else if (!real && !pred) vn++;
      else if (!real && pred) fp++;
      else fn++;
    }
    System.out.println("                    Predicho: Sí   Predicho: No");
    System.out.printf("Real: Sí (aprobar)       VP = %d        FN = %d%n", vp, fn);
    System.out.printf("Real: No (denegar)       FP = %d        VN = %d%n", fp, vn);

    int total = vp + vn + fp + fn;
    System.out.printf("%nExactitud = (VP + VN) / total = (%d + %d) / %d = %.1f %%%n",
        vp, vn, total, 100.0 * (vp + vn) / total);
    System.out.printf("Precisión = VP / (VP + FP)    = %d / %d = %.1f %%%n",
        vp, vp + fp, 100.0 * vp / (vp + fp));
    System.out.printf("Recall    = VP / (VP + FN)    = %d / %d = %.1f %%%n",
        vp, vp + fn, 100.0 * vp / (vp + fn));
  }
}
```

▶ Resultado esperado en la terminal

```
                    Predicho: Sí   Predicho: No
Real: Sí (aprobar)       VP = 3        FN = 0
Real: No (denegar)       FP = 1        VN = 2

Exactitud = (VP + VN) / total = (3 + 2) / 6 = 83.3 %
Precisión = VP / (VP + FP)    = 3 / 4 = 75.0 %
Recall    = VP / (VP + FN)    = 3 / 3 = 100.0 %
```

Paso 3. Respuesta explicada

- El único error fue un **falso positivo**: el cliente (Medio, Sí, Estable, Malo) debía ser denegado y el árbol lo aprobó.
- **Precisión 75 %:** de los 4 créditos que el sistema aprobó, 3 eran correctos. **Recall 100 %:** el sistema no rechazó a ningún buen cliente.

- **El error más costoso es el falso positivo:** aprobar un crédito a quien no podrá pagar significa **perder el dinero prestado**. Un falso negativo solo significa perder la ganancia de un cliente.
- Por eso en créditos se busca una **precisión alta**, aunque se rechacen algunos buenos clientes. La exactitud sola (83.3 %) no muestra qué tipo de error comete el modelo.

Respuestas a las preguntas de reflexión

1. ¿Por qué un atributo con ganancia 0 no sirve como pregunta?

Ganancia 0 significa que, después de separar los datos por ese atributo, la entropía ponderada **es igual** a la inicial: cada grupo tiene la misma proporción de clases que el conjunto completo. La pregunta no reduce la incertidumbre; es como preguntar algo cuya respuesta no ayuda a adivinar.

2. Relación de ID3 con la recursividad y los recorridos

ID3 construye el árbol de forma **recursiva**: `construir` tiene dos **casos base** (grupo puro o sin atributos) y se llama a sí mismo para cada rama. `clasificar` baja por el árbol recursivamente, como `buscar` en el árbol binario. `imprimir` e `imprimirReglas` son recorridos **pre-order**: primero el nodo y luego sus hijos. La diferencia es que los nodos de ID3 pueden tener **más de dos hijos**.

3. ¿Por qué un árbol que acierta el 100 % en entrenamiento podría fallar con datos nuevos?

Porque puede haber **memorizado** los datos de entrenamiento, incluidos casos raros o con errores, en lugar de aprender el patrón general: es el **sobreajuste**. ID3 sigue dividiendo hasta que cada hoja es pura, así que crea ramas muy específicas. En el Ejercicio 3, un solo día nuevo duplicó el número de reglas. Se evita con más datos, con la **poda** del árbol o limitando su profundidad.

4. ¿Por qué los árboles son modelos explicables?

Porque cada predicción se puede justificar con una **regla legible**: «Se denegó el crédito porque tiene deudas y su ingreso es bajo». Una red neuronal, en cambio, decide con miles de pesos difíciles de interpretar. Explicar las decisiones es importante cuando afectan a las personas: créditos, contratación de personal, diagnósticos médicos o decisiones legales. En muchos países la ley exige poder explicar por qué un sistema automático rechazó a alguien.